

Many ways to plot images

The most common way to plot images in Matplotlib is with `imshow`. The following examples demonstrate much of the functionality of `imshow` and the many images you can create.

```
import matplotlib.pyplot as plt
import numpy as np

import matplotlib.cbook as cbook
import matplotlib.cm as cm
from matplotlib.patches import PathPatch
from matplotlib.path import Path

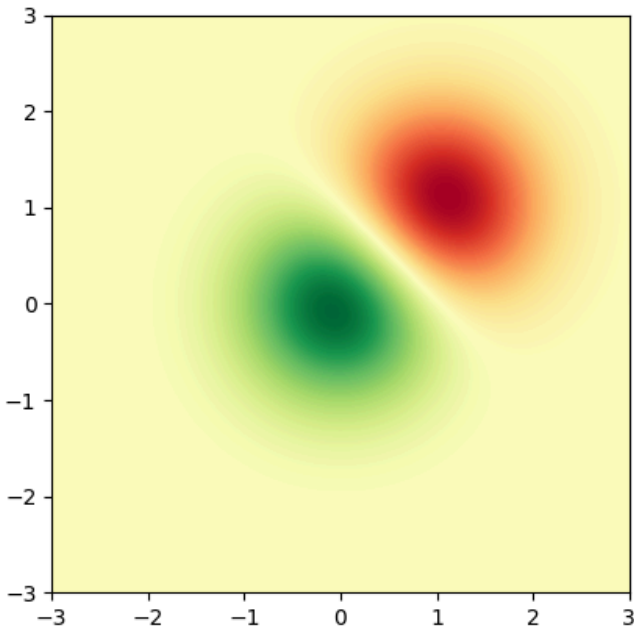
# Fixing random state for reproducibility
np.random.seed(19680801)
```

First we'll generate a simple bivariate normal distribution.

```
delta = 0.025
x = y = np.arange(-3.0, 3.0, delta)
X, Y = np.meshgrid(x, y)
Z1 = np.exp(-X**2 - Y**2)
Z2 = np.exp(-(X - 1)**2 - (Y - 1)**2)
Z = (Z1 - Z2) * 2

fig, ax = plt.subplots()
im = ax.imshow(Z, interpolation='bilinear', cmap=cm.RdYlGn,
               origin='lower', extent=[-3, 3, -3, 3],
               vmax=abs(Z).max(), vmin=-abs(Z).max())

plt.show()
```



It is also possible to show images of pictures.

```
# A sample image
with cbook.get_sample_data('grace_hopper.jpg') as image_file:
    image = plt.imread(image_file)

# And another image, using 256x256 16-bit integers.
w, h = 256, 256
with cbook.get_sample_data('s1045.ima.gz') as datafile:
    s = datafile.read()
A = np.frombuffer(s, np.uint16).astype(float).reshape((w, h))
extent = (0, 25, 0, 25)

fig, ax = plt.subplot_mosaic([
    ['hopper', 'mri']
], figsize=(7, 3.5))

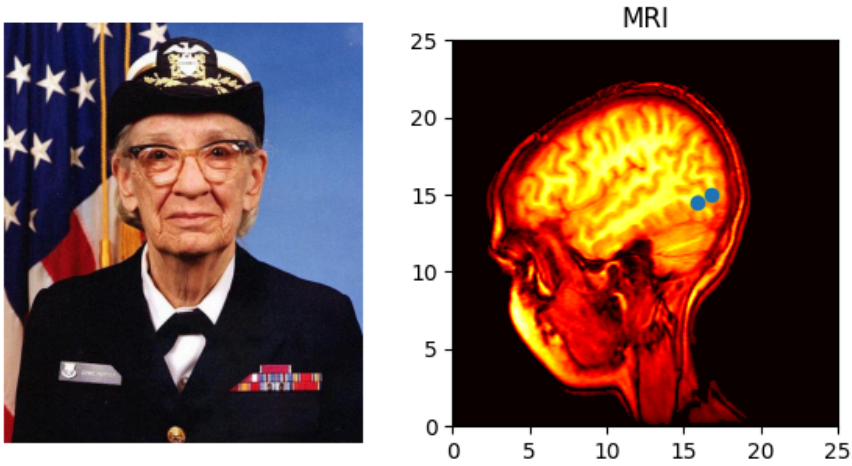
ax['hopper'].imshow(image)
ax['hopper'].axis('off') # clear x-axis and y-axis

im = ax['mri'].imshow(A, cmap=plt.cm.hot, origin='upper', extent=extent)

markers = [(15.9, 14.5), (16.8, 15)]
x, y = zip(*markers)
ax['mri'].plot(x, y, 'o')

ax['mri'].set_title('MRI')

plt.show()
```



Interpolating images

It is also possible to interpolate images before displaying them. Be careful, as this may manipulate the way your data looks, but it can be helpful for achieving the look you want. Below we'll display the same (small) array, interpolated with three different interpolation methods.

The center of the pixel at $A[i, j]$ is plotted at $(i+0.5, j+0.5)$. If you are using `interpolation='nearest'`, the region bounded by (i, j) and $(i+1, j+1)$ will have the same color. If you are using interpolation, the pixel center will have the same color as it does with nearest, but other pixels will be interpolated between the neighboring pixels.

To prevent edge effects when doing interpolation, Matplotlib pads the input array with identical pixels around the edge: if you have a 5x5 array with colors a-y as below:

```
a b c d e
f g h i j
k l m n o
p q r s t
u v w x y
```

Matplotlib computes the interpolation and resizing on the padded array

```
a a b c d e e
a a b c d e e
f f g h i j j
k k l m n o o
p p q r s t t
o o v w x y y
o o v w x y y
```

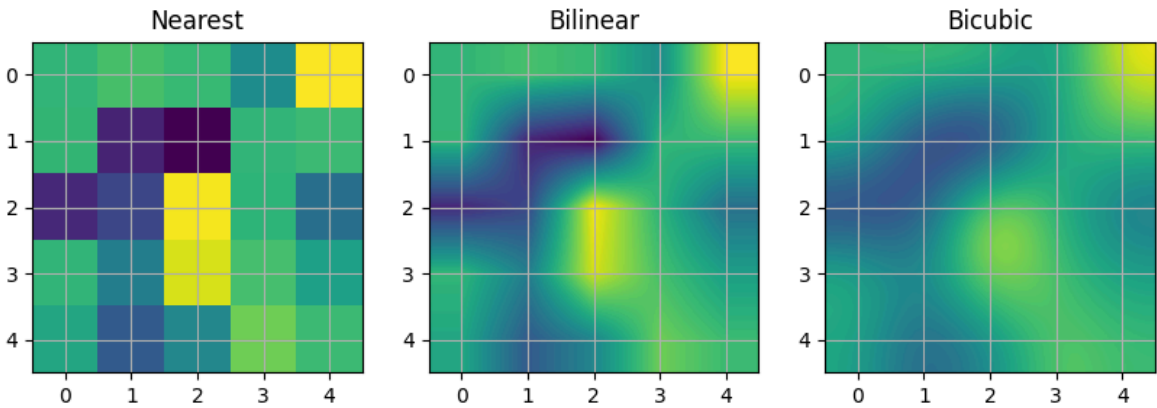
and then extracts the central region of the result. (Extremely old versions of Matplotlib (<0.63) did not pad the array, but instead adjusted the view limits to hide the affected edge areas.)

This approach allows plotting the full extent of an array without edge effects, and for example to layer multiple images of different sizes over one another with different interpolation methods -- see [Layer Images](#). It also implies a performance hit, as this new temporary, padded array must be created. Sophisticated interpolation also implies a performance hit; for maximal performance or very large images, `interpolation='nearest'` is suggested.

```
A = np.random.rand(5, 5)

fig, axs = plt.subplots(1, 3, figsize=(10, 3))
for ax, interp in zip(axs, ['nearest', 'bilinear', 'bicubic']):
    ax.imshow(A, interpolation=interp)
    ax.set_title(interp.capitalize())
    ax.grid(True)

plt.show()
```



You can specify whether images should be plotted with the array origin $x[0, 0]$ in the upper left or lower right by using the `origin` parameter. You can also control the default setting `image.origin` in your [matplotlibrc file](#). For more on this topic see the [complete guide on origin and extent](#).

- Colormap normalizations SymLogNorm
- Contour Corner Mask
- Contour Demo
- Contour Image
- Contour Label Demo
- Contourf demo
- Contourf Hatching
- Contourf and log color scale
- Contouring the solution space of optimizations
- BboxImage Demo
- Figimage Demo
- Creating annotated heatmaps
- Image antialiasing
- Clipping images with patches
- Many ways to plot images**
- Image Masked
- Image nonuniform
- Blend transparency with color in 2D images
- Modifying the coordinate formatter
- Interpolations for imshow

interpolations for imshow

Contour plot of irregularly spaced data

Layer Images

Visualize matrices with matshow

Multiple images with one colorbar

pcolor images

pcolormesh grids and shading

pcolormesh

Streamplot

QuadMesh Demo

Advanced quiver and quiverkey functions

Quiver Simple Demo

Shading example

Spectrogram

Spy Demos

Tricontour Demo

Tricontour Smooth Delaunay

Tricontour Smooth User

Trigradient Demo

Triinterp Demo

Tripcolor Demo

Triplot Demo

Watermark image

Subplots, axes and figures

Statistics

Pie and polar charts

Text, labels and annotations

Color

Shapes and collections

Style sheets

Module - pyplot

Module - axes_grid1

Module - axisartist

Showcase

Animation

Event handling

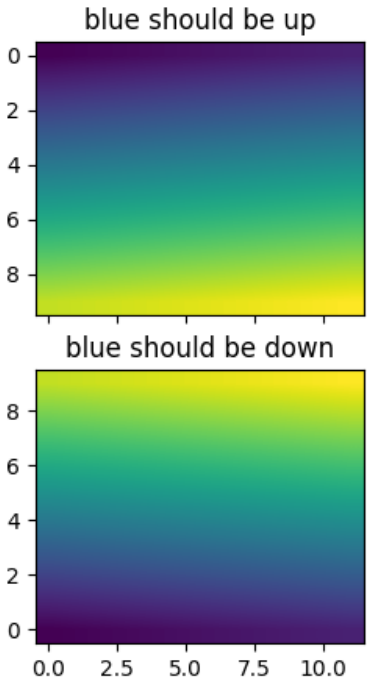
Miscellaneous

3D plotting

```
x = np.arange(120).reshape((10, 12))

interp = 'bilinear'
fig, axs = plt.subplots(nrows=2, sharex=True, figsize=(3, 5))
axs[0].set_title('blue should be up')
axs[0].imshow(x, origin='upper', interpolation=interp)

axs[1].set_title('blue should be down')
axs[1].imshow(x, origin='lower', interpolation=interp)
plt.show()
```



Finally, we'll show an image using a clip path.

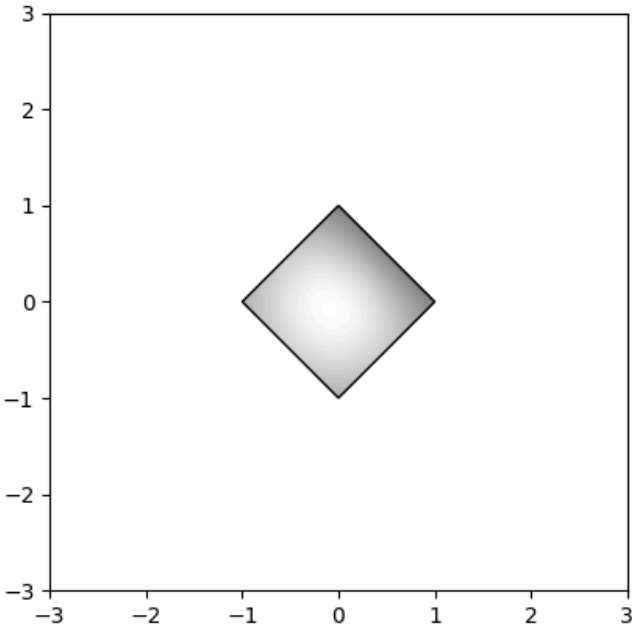
```
delta = 0.025
x = y = np.arange(-3.0, 3.0, delta)
X, Y = np.meshgrid(x, y)
Z1 = np.exp(-X**2 - Y**2)
Z2 = np.exp(-(X - 1)**2 - (Y - 1)**2)
Z = (Z1 - Z2) * 2

path = Path([[0, 1], [1, 0], [0, -1], [-1, 0], [0, 1]])
patch = PathPatch(path, facecolor='none')

fig, ax = plt.subplots()
ax.add_patch(patch)

im = ax.imshow(Z, interpolation='bilinear', cmap=cm.gray,
               origin='lower', extent=[-3, 3, -3, 3],
               clip_path=patch, clip_on=True)
im.set_clip_path(patch)

plt.show()
```



References

The use of the following functions, methods, classes and modules is shown in this example:

- `matplotlib.axes.Axes.imshow` / `matplotlib.pyplot.imshow`
- `matplotlib.artist.Artist.set_clip_path`
- `matplotlib.patches.PathPatch`

© Copyright 2002–2012 John Hunter, Darren Dale, Eric Firing, Michael Droettboom and the Matplotlib development team; 2012–2024 The Matplotlib development team.

Created using [Sphinx](#) 8.0.2.

Built from v3.9.2-3-gd04b2f64fe.

Built with the [PyData Sphinx Theme](#)
0.15.4.